

Session 1: Introduction to Operating Systems (OS)

1. What is an Operating System (OS)?

- **Definition:** An Operating System (OS) is system software that manages hardware and software resources on a computer. It acts as an intermediary between users and the hardware to ensure the efficient operation of the machine.
 - **Difference from Application Software:**
 - **Application Software:** Programs designed for specific tasks (e.g., Microsoft Word for document creation).
 - **Operating System:** Manages hardware resources and provides a platform for running application software.
 - Example: An OS controls how Word interacts with the CPU, memory, and printer, but Word itself doesn't interact directly with the hardware.
 - **Why is the OS Hardware Dependent?**
 - Every OS is designed to work with specific hardware architectures. For example:
 - **Windows OS:** Works with x86 and x64 architectures.
 - **macOS:** Built for Apple hardware (ARM-based M1/M2 or Intel in older versions).
 - **Android:** Primarily designed for ARM architecture in mobile devices.
-

2. Different Components of an OS

The OS comprises multiple components, each with specific functions:

- **Kernel:** The core of the OS that directly interacts with hardware. It manages resources like CPU, memory, and devices.
 - Example: Linux Kernel.
 - **File System:** Manages data storage, retrieval, and organization on storage devices.
 - Example: NTFS in Windows, ext4 in Linux.
 - **Process Management:** Manages execution of processes, their states, and multitasking.
 - Example: Handling multiple applications running simultaneously.
 - **Memory Management:** Allocates and deallocates memory to processes.
 - Example: Virtual memory management in Windows.
 - **Device Management:** Manages device communication via drivers.
 - Example: Printer drivers.
 - **User Interface (UI):** Provides a user-friendly interface to interact with the OS.
 - Example: GUI in Windows/macOS, CLI in Linux.
-

3. Basic Computer Organization for OS

To understand OS functionality, it's crucial to understand the basic components of a computer:

- **Central Processing Unit (CPU):** Executes instructions and processes data.
- **Memory (RAM):** Temporarily holds data and instructions for quick access.
- **Input/Output (I/O) Devices:** Interfaces like keyboards, mice, monitors, printers.
- **Storage Devices:** Hard drives, SSDs, used for long-term data storage.

The OS coordinates these components, ensuring smooth interaction between hardware and software.

4. Examples of Well-Known OS and Their Types

1. Desktop OS:

- Examples: Windows, macOS, Linux (Ubuntu).
- **Purpose:** Designed for general-purpose computing (home, office).

2. Mobile OS:

- Examples: Android, iOS.
- **Purpose:** Optimized for touchscreen devices and mobile hardware constraints.

3. Embedded System OS:

- Examples: VxWorks, FreeRTOS.
- **Purpose:** Embedded in devices like washing machines, routers, and cars. Highly specialized and lightweight.

4. Real-Time OS (RTOS):

- Examples: QNX, RTEMS.
- **Purpose:** Ensures tasks are executed within strict time constraints. Common in avionics and medical systems.

5. Server OS:

- Examples: Windows Server, Red Hat Enterprise Linux.
- **Purpose:** Optimized for managing server hardware and handling multiple users.

6. Hybrid OS:

- Examples: Android (Linux-based kernel with a user-friendly UI).
 - **Purpose:** Combines features of multiple OS types.
-

5. Functions of an OS

The primary functions of an OS include:

1. Process Management:

- Controls process creation, scheduling, and termination.
- Example: Running a browser and a music player simultaneously.

2. Memory Management:

- Allocates and tracks memory usage by programs.
- Example: Virtual memory extends physical memory using storage.

3. File Management:

- Manages file creation, storage, retrieval, and permissions.
 - Example: Organizing files in folders.
4. **Device Management:**
- Coordinates input/output devices via drivers.
 - Example: Recognizing a USB drive.
5. **Security and Access Control:**
- Protects data and resources through authentication and permissions.
 - Example: Login credentials.
6. **Networking:**
- Enables communication between devices over a network.
 - Example: Sharing files over Wi-Fi.
7. **Error Detection and Handling:**
- Detects and resolves hardware and software errors.
 - Example: Notifying about a faulty hard disk.
-

6. User and Kernel Space; Modes

- **User Space:**
 - Where user applications run.
 - Applications interact with the OS using system calls.
 - Example: A media player running in user space.
 - **Kernel Space:**
 - Reserved for the core OS functions.
 - Example: Memory management or device control.
 - **Modes:**
 - **User Mode:** Restricted access to hardware for safety.
 - **Kernel Mode:** Full access to hardware, used by the OS kernel.
-

7. Interrupts and System Calls

- **Interrupts:**
 - Signals that alert the CPU to immediate tasks.
 - Example: A keyboard press sends an interrupt to the CPU.
 - **System Calls:**
 - Requests made by programs in user space for OS services.
 - Example: Reading a file involves a system call to the OS for accessing the file system.
-

By understanding these points, you can grasp the foundational concepts of Operating Systems and their critical role in computing systems.

